Comparison of Lightweight Encryption Algorithms

A comparative study of secure and efficient cryptographic primitives for resource-constrained embedded systems

Technical Writing Report Submitted

In partial fulfillment of the requirements for the award of the

Bachelor of Technology

Authors

Rajesh Kumar Jena (B122088)

Soumil Kumar (B122114)

Punit Kumar (B422041)

Under the supervision of

Prof. Debasish Jena

International Institute of Information Technology, Bhubaneswar

November 2025

Approval of the Viva-Voce Board

November 18, 2025

Certified that the report entitled "Comparison of Lightweight Encryption Algorithms," submitted by Rajesh Kumar Jena (B122088), Soumil Kumar (B122114), and Punit Kumar (B422041) to the International Institute of Information Technology, Bhubaneswar, in partial fulfillment of the Technical Writing requirements for the award of the Bachelor of Technology under the B.Tech Programme, has been accepted by the examiners during the viva-voce examination held today.

Prof. Tushar Ranjan Sahoo

Prof. Sabyasachi Patra

Prof. Srichandan Sobhanayak

Prof. Debasish Jena

Certificate

This is to certify that the report entitled "Comparison of Lightweight Encryption Algorithms," submitted by Rajesh Kumar Jena (B122088), Soumil Kumar (B122114), and Punit Kumar (B422041) to the International Institute of Information Technology, Bhubaneswar, is a record of bona fide work carried out under my supervision. The report is submitted for the end-semester evaluation of the B.Tech Technical Writing course.

Prof. Debasish Jena

Declaration

We hereby declare that:

1. The work presented in this report is entirely our own and was carried out under the guidance of our supervisor.
2. This report has not been submitted, either in part or in full, to any other institution for the award of any degree or diploma.
3. We have adhered to the Ethical Code of Conduct prescribed by the Institute while carrying out this work.
4. Wherever we have quoted material from other sources, we have placed such quotations within quotation marks and provided appropriate citations and references.
5. Whenever we have used ideas, data, analysis, or text derived from other sources, we have duly acknowledged them within the report and included full details in the reference section.
6. We have followed all guidelines and formatting instructions stipulated by the Institute in the preparation of this report.

Rajesh Kumar Jena	**B122088**
Soumil Kumar	**B122114**
Punit Kumar	**B422041**

Acknowledgments

We express our sincere gratitude to our supervisor, Prof. Debasish Jena, for his invaluable guidance, continuous support, and insightful feedback throughout this project. His expertise was instrumental in helping us navigate challenges and refine our work.

We also thank the faculty and staff of the Department of Computer Science and Engineering and the Department of Information Technology at the International Institute of Information Technology, Bhubaneswar, for providing the resources, infrastructure, and academic environment necessary for the successful completion of this project.

Finally, we extend our heartfelt appreciation to our families and friends for their constant support and understanding throughout the duration of this work.

Rajesh Kumar Jena
Soumil Kumar
Punit Kumar

Abstract

The proliferation of resource-constrained devices in IoT networks, embedded controllers, and sensor systems has increased the need for lightweight cryptographic solutions. Conventional encryption algorithms often exceed the computational capabilities, memory constraints, and energy budgets of such platforms. This study presents a comparative analysis of three prominent lightweight encryption algorithms—PRESENT, SPECK, and ASCON—and evaluates their suitability for constrained embedded environments.

The methodology combines theoretical analysis of design principles with empirical performance benchmarking on a Raspberry Pi Pico microcontroller using Rust implementations. Metrics, including execution time, memory utilization, throughput, and structural security properties, are systematically evaluated.

The findings aim to provide a clearer understanding of how lightweight ciphers balance security and efficiency in practical embedded systems.

Keywords: lightweight cryptography, embedded systems, PRESENT, SPECK, ASCON, IoT security, performance evaluation

Contents

Approval of the Viva-Voce Board **1**

Certificate **2**

Declaration **3**

Acknowledgments **4**

Abstract **5**

1 Introduction **8**

1.1 Historical Context and Background 8

1.2 Motivation: Computing at the Edge 8

1.3 Problem Statement 9

1.4 Objectives 9

2 Literature Survey **11**

2.1 Introduction to Lightweight Cryptography 11

2.2 Categories of Lightweight Cryptographic Algorithms 12

2.2.1 Lightweight Block Ciphers 12

2.2.2 Lightweight Stream Ciphers 13

2.2.3 Lightweight Hash Functions 13

2.2.4 Lightweight Authenticated Encryption 13

2.3 Selected Algorithms for Evaluation 13

2.3.1 PRESENT Cipher 14

2.3.2 SIMON Cipher (Reference Algorithm) 14

2.3.3 SPECK Cipher 14

2.3.4 ASCON Cipher 14

3 Algorithm Overview **15**
3.1 PRESENT 15
3.2 SPECK 17
3.3 ASCON 18

4 Proposed Comparative Framework and Methodology **20**
4.1 Evaluation Criteria 20
4.1.1 Performance Metrics 20
4.1.2 Security Metrics 21
4.2 Algorithm Selection 22
4.3 Implementation Plan 23
4.3.1 Software and Hardware Environment 23
4.3.2 Performance Measurement Procedure 23
4.3.3 Memory Profiling Procedure 23
4.4 Summary of Methodology 23

5 Expected Results and Discussion **24**
5.1 Expected Performance Results 24
5.1.1 Execution Time 24
5.1.2 Throughput 24
5.2 Expected Memory Utilization 25
5.3 Expected Security Observations 25
5.4 Illustrative Result Tables 26
5.5 Discussion 26

6 Conclusion and Future Work **27**
6.1 Conclusion 27
6.2 Future Work 27

1. Introduction

1.1 Historical Context and Background

Cryptography has supported secure communication for thousands of years, beginning with simple manual techniques used by ancient civilizations. Early methods such as the Spartan *scytale* (5th century BCE) applied basic transposition to conceal messages [1]. Similar substitution-based techniques appeared in the Indian *Arthashastra* and Chinese administrative systems [1]. Although these schemes relied primarily on obscurity, they established foundational principles for secure communication.

A major advancement occurred in the medieval period when Al-Kindi introduced frequency analysis (9th century), marking the first systematic approach to cryptanalysis [2]. Renaissance-era developments, such as Alberti's polyalphabetic cipher, further strengthened resistance to statistical attacks [1].

In the 20th century, electromechanical cipher machines such as the Enigma dramatically expanded the practical key space. The successful cryptanalysis of Enigma at Bletchley Park demonstrated the increasing importance of computation in cryptography and is often recognized as a milestone in the evolution of modern cryptanalysis [3].

Modern cryptography emerged in the late 20th century with standardized symmetric- and public-key systems. AES provided robust symmetric-key encryption [4], while Diffie–Hellman key exchange [5] and RSA [6] enabled secure communication over untrusted networks.

1.2 Motivation: Computing at the Edge

The rapid expansion of Internet of Things (IoT) deployments has resulted in billions of devices operating under strict constraints on processing capability, memory capacity, and energy resources. Traditional cryptographic algorithms, designed for powerful general-purpose systems, frequently prove impractical in such environments.

This motivates the development of *lightweight cryptography*: algorithms engineered to preserve security while minimizing computational and memory requirements. Lightweight designs emphasize:

- minimization of computational complexity and cycle counts,
- reduction of RAM and ROM footprint,
- efficient operation on low-power hardware, and
- maintenance of practical security levels (typically 80–128 bits).

Accordingly, this investigation explores lightweight encryption algorithms as effective solutions for securing communication on embedded devices such as the Raspberry Pi Pico.

1.3 Problem Statement

Conventional algorithms such as AES, RSA, and ECC offer strong security guarantees but were developed with the assumption of abundant computational and memory resources. Their requirements often exceed the capabilities of microcontrollers commonly used in IoT devices [16].

Devices with limited processing frequencies, small RAM capacities, and stringent power budgets struggle to run cryptographic routines that depend on large lookup tables, multi-precision arithmetic, or high cycle counts [16]. As a result, developers sometimes deploy weakened, partial, or ad hoc security mechanisms, leading to vulnerabilities such as unencrypted communication, reused keys, and insufficient authentication. Incidents such as the Mirai botnet attack highlight the risks of insecure embedded systems.

Lightweight cryptography addresses this gap by offering efficient primitives tailored to constrained platforms without compromising core security goals.

1.4 Objectives

This report aims to:

- examine the evolution and design principles of lightweight cryptographic algorithms,
- evaluate selected lightweight ciphers (PRESENT, SPECK, ASCON) in terms of execution time, memory usage, and implementation cost,

- analyze their security properties based on established cryptanalytic research,
- implement and benchmark these algorithms on a Raspberry Pi Pico using Rust [18], and
- identify design trade-offs and assess their suitability for real-world constrained environments.

2. Literature Survey

2.1 Introduction to Lightweight Cryptography

Lightweight cryptography focuses on designing secure primitives that operate efficiently under strict computational, memory, and energy constraints in embedded and IoT devices [7, 8]. Unlike traditional algorithms such as AES or RSA, which assume powerful processors and ample memory, lightweight designs must balance three primary goals: security, performance, and implementation cost.

As billions of low-power devices become interconnected, the need for algorithms tailored to constrained environments has become critical. Lightweight primitives achieve this by minimizing round complexity, reducing memory usage, and avoiding expensive arithmetic operations [11].

Defining Lightweight Characteristics

Lightweight algorithms typically exhibit:

- **Computational efficiency**: reliance on simple operations such as XOR, bit rotations, and small S-box lookups,
- **Small memory footprint**: targeting less than 2 KB of Flash and under 1 KB of RAM to accommodate compact microcontrollers and RFID-class devices,
- **Energy efficiency**: avoiding expensive operations (e.g., modular arithmetic, large lookup tables) to prolong battery life in constrained platforms.

These features differentiate lightweight ciphers from conventional symmetric- and public-key algorithms, which typically exceed the capabilities of embedded hardware.

The Lightweight Design Trilemma

The development of lightweight ciphers involves a fundamental design trade-off among three aspects:

- **Security**: maintaining resistance to differential, linear, and structural attacks at practical security levels,
- **Performance**: enabling real-time operation on 8-bit and 32-bit microcontrollers, and
- **Implementation cost**: minimizing area in hardware, code size in software, and RAM usage.

Balancing these factors requires careful design of round functions, diffusion mechanisms, and key schedules.

2.2 Categories of Lightweight Cryptographic Algorithms

Lightweight designs span multiple cryptographic categories. The following represent the most relevant primitives for constrained platforms.

2.2.1 Lightweight Block Ciphers

Block ciphers remain the most widely studied category due to their versatility and well-understood security models. Representative designs include:

- **PRESENT**: a hardware-oriented 64-bit SPN cipher known for extremely low gate count [7],
- **SIMON & SPECK**: NSA-designed lightweight ciphers optimized for hardware (SIMON) and software (SPECK) [8],
- **CLEFIA**: a Sony-designed Feistel cipher offering strong diffusion with efficient hardware implementation.

2.2.2 Lightweight Stream Ciphers

Stream ciphers generate a keystream for encrypting data, making them suitable for high-throughput and low-latency applications. Examples include:

- **GRAIN-128a**: built using LFSR and NFSR components [14],
- **TRIVIUM**: an eSTREAM finalist with a compact and efficient structure [15].

2.2.3 Lightweight Hash Functions

Lightweight hash functions support authentication and integrity in constrained environments. Notable designs include:

- **PHOTON**: an AES-inspired design [12],
- **SPONGENT**: a sponge-based lightweight hash [13].

2.2.4 Lightweight Authenticated Encryption

Authenticated encryption combines confidentiality and integrity in a single primitive. The most significant modern lightweight AEAD design is:

- **ASCON**: the NIST Lightweight Cryptography Standard (2023), offering efficient authenticated encryption and hashing across hardware and software platforms [9, 10].

ASCON's balance of security, efficiency, and simplicity makes it a leading candidate for future IoT deployments.

2.3 Selected Algorithms for Evaluation

This study focuses on three lightweight ciphers selected for practical implementation and benchmarking on the Raspberry Pi Pico: PRESENT, SPECK, and ASCON. These algorithms represent distinct design philosophies spanning both classical and modern approaches to lightweight cryptography.

2.3.1 PRESENT Cipher

PRESENT is a compact 64-bit SPN block cipher designed for highly constrained hardware. Its 4-bit S-box and bit-permutation layer minimize gate count, making it suitable for RFID tags, sensor nodes, and smart cards [7]. Although not optimized for high software speed, its simplicity keeps its resource usage minimal.

2.3.2 SIMON Cipher (Reference Algorithm)

SIMON uses a Feistel network with bitwise AND, XOR, and rotation operations, resulting in extremely compact hardware implementations [8]. Due to concerns about NSA involvement, SIMON is not implemented in this project but is included for literature completeness.

2.3.3 SPECK Cipher

SPECK is optimized for software using an ARX (Add-Rotate-XOR) structure. Its round function is compact, efficient, and achieves strong performance on general-purpose microcontrollers [8]. Despite debates regarding its origin, SPECK remains one of the fastest lightweight block ciphers reported in the literature.

2.3.4 ASCON Cipher

ASCON represents the modern generation of lightweight cryptography. Based on a sponge-duplex construction, it supports authenticated encryption and hashing with a 320-bit internal state. Its balance of security and efficiency led to its selection as the NIST Lightweight Cryptography Standard [9, 10]. It is suited for firmware authentication, IoT communications, and embedded control systems.

This literature review establishes the foundation for evaluating PRESENT, SPECK, and ASCON in subsequent chapters through practical benchmarking and comparative analysis.

3. Algorithm Overview

This chapter provides a concise overview of the three lightweight cryptographic algorithms evaluated in this report: PRESENT, SPECK, and ASCON. For each algorithm, we summarize its design structure, illustrate its high-level block diagram, and present simplified pseudocode for the encryption or authenticated-encryption process.

3.1 PRESENT

Structure and Design Goals

PRESENT is a 64-bit substitution–permutation network (SPN) block cipher with key sizes of 80 or 128 bits [7]. It was designed for ultra-lightweight hardware implementations with a very low gate count. Each round consists of an AddRoundKey step, a non-linear substitution layer using 4-bit S-boxes, and a bit-permutation layer that provides diffusion. A simple key schedule generates round keys from the master key.

Round Structure Diagram

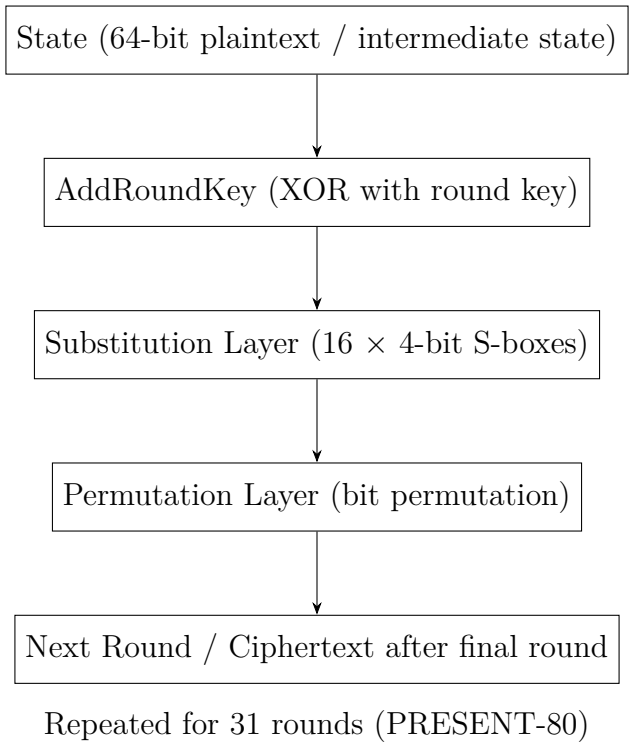


Figure 3.1: Simplified round structure of the PRESENT block cipher.

High-Level Encryption Pseudocode

Algorithm 1 PRESENT Encryption (High-Level)

Require: 64-bit plaintext state S, master key K
Ensure: 64-bit ciphertext C

```
Generate round keys {K_1, K_2, ..., K_31} from K
for i = 1 to 31 do
    S ← S ⊕ K_i                                  {AddRoundKey}
    S ← SBoxLayer(S)
    S ← PermutationLayer(S)
end for
C ← S ⊕ K_32                                     {Final whitening (if used)}
return C
```

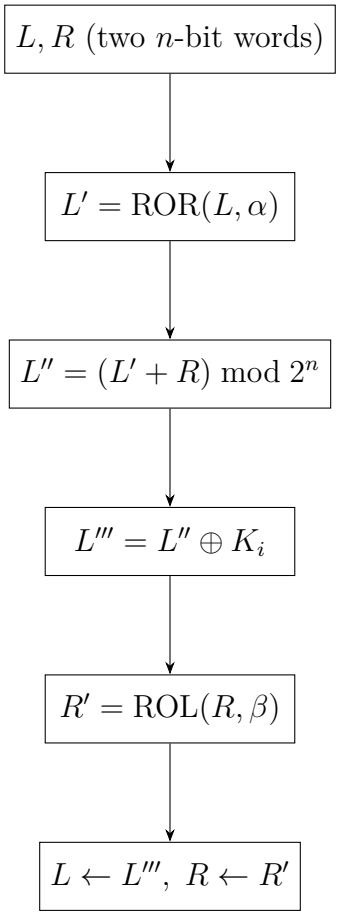
The pseudocode above omits implementation details of the key schedule, S-box definitions, and bit permutation, focusing instead on the round-level structure.

3.2 SPECK

Structure and Design Goals

SPECK is a family of lightweight block ciphers optimized for software using an ARX (Add–Rotate–XOR) design [8]. It uses a simple Feistel-like structure with word-wise additions modulo 2^n, rotations, and XOR operations. This makes SPECK highly efficient on microcontrollers, particularly in software-only environments.

Round Function Diagram



Repeated for a fixed number of rounds (e.g., 26 for SPECK-64/128).

Figure 3.2: Simplified round of the SPECK block cipher (ARX Feistel-like structure).

High-Level Encryption Pseudocode

Algorithm 2 SPECK Encryption (High-Level)

Require: Two n-bit words (L, R) as plaintext, round keys $\{K_0, \ldots, K_{T-1}\}$
Ensure: Two n-bit words (L, R) as ciphertext

```
for i = 0 to T − 1 do
    L ← ROR(L, α)
    L ← (L + R) mod 2^n
    L ← L ⊕ K_i
    R ← ROL(R, β)
    R ← R ⊕ L
end for
return (L, R)
```

Here, α and β are rotation constants determined by the chosen SPECK variant (e.g., SPECK-64/128).

3.3 ASCON

Structure and Design Goals

ASCON is a family of lightweight authenticated encryption and hashing algorithms based on a permutation over a 320-bit internal state [9, 10]. The state is typically viewed as five 64-bit words. ASCON uses a sponge-duplex construction, where data is absorbed into part of the state (the rate), interleaved with permutation calls, and then ciphertext and authentication tags are produced by squeezing from the same state.

Sponge-Duplex Overview

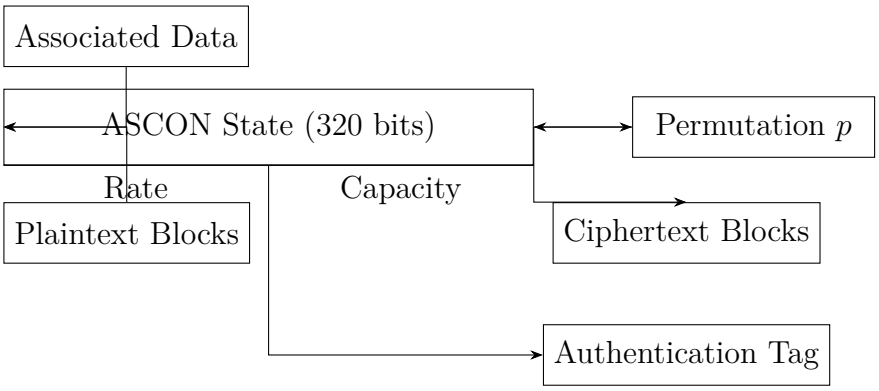


Figure 3.3: High-level view of ASCON authenticated-encryption using a sponge-duplex construction.

High-Level AEAD Pseudocode

Algorithm 3 ASCON AEAD Encryption (High-Level)

Require: Key K, nonce N, associated data A, plaintext P
Ensure: Ciphertext C, authentication tag T

```
Initialize 320-bit state with K, N, and IV
Apply initial permutation p
Absorb associated data:
for each block A_i of A do
    Absorb A_i into the rate portion of the state
    Apply permutation p
end for
Encrypt plaintext and produce ciphertext:
for each block P_i of P do
    Absorb P_i into the rate portion of the state
    C_i ← part of the state (rate) XOR P_i
    Apply permutation p
end for
Finalize and generate tag:
Inject key material into the state
Apply final permutation p
Derive tag T from part of the state
return (C, T)
```

This pseudocode abstracts away implementation-specific details, such as padding rules, exact rate/capacity split, and permutation round counts, but it captures the main structure of ASCON's authenticated-encryption mode.

4. Proposed Comparative Framework and Methodology

This chapter presents the methodology used to evaluate the lightweight cryptographic algorithms—PRESENT, SPECK, and ASCON—on an embedded platform. The objective is to enable a fair, reproducible, and practically feasible comparison using software-based instrumentation on the Raspberry Pi Pico [17]. The methodology covers performance measurements, memory analysis, security considerations, and the rationale behind algorithm selection.

4.1 Evaluation Criteria

Lightweight cryptographic algorithms must be assessed along dimensions relevant to constrained devices. The evaluation criteria integrate quantitative measurements from Rust implementations with qualitative analysis based on cryptographic literature.

4.1.1 Performance Metrics

Performance evaluation focuses on execution efficiency on the ARM Cortex-M0+ microcontroller.

Execution Time

Execution time measurements include:

- Key schedule generation,
- Single-block encryption and decryption,
- Full-message encryption for 64 B, 256 B, and 1024 B inputs.

Timing measurements utilize the hardware timer available on the Raspberry Pi Pico via the `rp2040-hal` crate. Cycle counts are calculated as:

$$cycles = time\ in\ seconds \times 133\ \text{MHz}.$$

Cycles Per Byte (CPB)

CPB, a platform-independent indicator of efficiency, is computed as:

$$CPB = \frac{total\ cycles}{total\ bytes\ processed}.$$

Memory Utilization

Memory analysis records:

- **ROM footprint**: compiled binary size (`cargo size -release`),
- **Static RAM usage**: ELF metadata (`.bss` and `.data` sections),
- **Working RAM**: estimated from internal state, S-boxes, and buffers.

Throughput

Throughput (bytes/s) is measured by encrypting buffers of fixed sizes:

$$throughput = \frac{bytes\ processed}{time}.$$

This metric highlights scaling behavior, particularly for ASCON's authenticated mode.

4.1.2 Security Metrics

Security analysis is based on literature, focusing on theoretical strength and resilience against practical attacks.

Security Level

Comparison includes:

- Key sizes (80-bit, 128-bit),
- Block sizes and internal state dimensions,
- Expected brute-force resistance.

Cryptanalytic Resistance

The study summarizes known attacks, including differential and linear cryptanalysis, meet-in-the-middle attacks, algebraic attacks, and rotational attacks (notably relevant for ARX-based ciphers such as SPECK).

Structural Robustness

A structural comparison evaluates:

- PRESENT's S-box and bit-permutation layer,
- SPECK's ARX-based Feistel structure,
- ASCON's sponge-based permutation.

4.2 Algorithm Selection

The selected algorithms represent diverse lightweight design approaches and are practical for implementation on the Raspberry Pi Pico.

PRESENT

A hardware-oriented SPN cipher standardized in ISO/IEC 29192-2. Its small hardware footprint makes it a representative lightweight cipher for constrained devices.

SPECK

A software-optimized ARX cipher offering high speed, low RAM usage, and compact code size. It maps efficiently onto the Pico's Cortex-M0+ instruction set.

ASCON

The NIST Lightweight Cryptography Standard (2023), providing authenticated encryption with balanced performance in hardware and software.

Together, these algorithms provide varied design philosophies while keeping the project scope manageable.

4.3 Implementation Plan

All experiments are conducted using Rust on the Raspberry Pi Pico to ensure practical relevance and feasibility within the project timeline.

4.3.1 Software and Hardware Environment

- **Hardware**: Raspberry Pi Pico (ARM Cortex-M0+, 133 MHz, 264 KB SRAM, 2 MB Flash),
- **Language**: Rust (`no_std` environment for embedded targets),
- **Tools**: `rp2040-hal`, `probe-rs`, `cargo-embed`,
- **Build Profile**: Release mode with LTO and size optimization.

4.3.2 Performance Measurement Procedure

- Use the hardware timer for cycle-accurate timing,
- Average results over at least 1000 iterations,
- Measure both single-block and multi-block performance,
- For ASCON, evaluate initialization, associated-data processing, encryption, and tag generation.

4.3.3 Memory Profiling Procedure

- Extract ROM and RAM usage from build artifacts,
- Identify memory-heavy components such as S-box tables or state buffers.

4.4 Summary of Methodology

This methodology enables a fair and meaningful comparison of PRESENT, SPECK, and ASCON within the context of lightweight cryptography for constrained embedded systems.

5. Expected Results and Discussion

This chapter outlines the expected outcomes from the implementation and benchmarking of the PRESENT, SPECK, and ASCON algorithms on the Raspberry Pi Pico platform. Since the actual experimental results will be obtained during the implementation phase, this chapter provides a discussion of anticipated trends based on published literature and the design characteristics of each algorithm.

5.1 Expected Performance Results

Algorithm performance on constrained microcontrollers is influenced by internal structure, operation complexity, and memory requirements. Based on published evaluations and design characteristics, the following performance trends are anticipated.

5.1.1 Execution Time

- **SPECK** is expected to provide the fastest execution time. Its ARX-based structure maps efficiently to the Cortex-M0+ instruction set, resulting in low cycles per round.
- **PRESENT** is expected to exhibit slower performance due to S-box lookups and bit-permutation operations that introduce software overhead.
- **ASCON** is expected to perform between PRESENT and SPECK. Its larger internal state (320 bits) and authenticated encryption requirements increase computational cost.

These trends should be observable in timing and cycle-per-byte (CPB) measurements.

5.1.2 Throughput

- **SPECK** is expected to achieve the highest throughput across different input sizes due to its streamlined round function.

- **ASCON** may demonstrate improved throughput for longer messages, as its initialization cost becomes amortized over larger inputs.
- **PRESENT** is expected to provide the lowest throughput due to its bitwise round transformations.

$$\text{Throughput (expected): SPECK} > \text{ASCON} > \text{PRESENT.}$$

5.2 Expected Memory Utilization

ROM (Binary Size): SPECK is expected to require the smallest ROM footprint because it relies exclusively on simple arithmetic and bitwise operations. PRESENT may require additional storage for S-box tables, but should remain compact overall. ASCON is expected to have the highest ROM usage due to its larger state and more complex permutations.

RAM Usage: PRESENT and SPECK are expected to use minimal static and working RAM. ASCON is expected to consume more RAM because it maintains a 320-bit internal state and additional temporary structures for authenticated encryption.

$$\text{RAM (expected): PRESENT} \approx \text{SPECK} < \text{ASCON.}$$

5.3 Expected Security Observations

All three algorithms provide practical levels of security for lightweight cryptography, but their assurances differ:

- **ASCON** offers the strongest modern security guarantees and was selected as the NIST Lightweight Cryptography Standard.
- **PRESENT-128** provides suitable margins for long-term embedded deployments, whereas PRESENT-80 is more appropriate for short-lifetime applications.
- **SPECK** maintains good cryptanalytic resistance but has tighter margins and continues to receive scrutiny due to its origins.

5.4 Illustrative Result Tables

The following table is a placeholder that will be populated with empirical results in the final report.

Algorithm	**Expected Speed**	**Expected ROM**	**Expected RAM**
PRESENT	Medium	Medium	Low
SPECK	High	Low	Low
ASCON	Medium	High	Medium

Table 5.1: Anticipated performance and memory-efficiency trends.

5.5 Discussion

Based on expected trends, SPECK is anticipated to deliver the best performance in software, making it suitable for real-time microcontroller applications. PRESENT, while slower, provides exceptional compactness that benefits low-cost hardware platforms. ASCON is expected to deliver a favorable balance of modern security and acceptable performance, particularly for systems requiring authenticated encryption.

These predictions reinforce a core principle of lightweight cryptography: different algorithms are optimized for different constraints, and therefore no single design dominates across all dimensions.

The empirical results obtained during experimentation will validate or refine these expectations and provide deeper insight into real-world suitability for embedded deployment.

6. Conclusion and Future Work

6.1 Conclusion

This study investigates lightweight cryptographic algorithms tailored for resource-constrained embedded platforms. A detailed literature review contextualizes the evolution of modern cryptography and highlights the emergence of lightweight approaches driven by IoT deployment challenges.

Three representative algorithms—PRESENT, SPECK, and ASCON—were selected to evaluate performance, memory efficiency, and theoretical security strength. The proposed methodology enables a structured comparison using Rust-based implementations on the Raspberry Pi Pico.

Expected trends suggest that SPECK will provide superior performance, PRESENT will offer minimal memory footprint, and ASCON will deliver strong modern security with functional completeness. Algorithm selection in embedded systems must therefore align with specific application needs and deployment constraints.

This report establishes a foundation for empirical benchmarking and provides methodological guidelines for further work.

6.2 Future Work

Future research can extend this study in the following directions:

- **Energy consumption analysis**: Measuring power efficiency during encryption and decryption.
- **Evaluation of additional ciphers**: Including designs such as SIMON, GIFT, and PRINCE across multiple variants.
- **Hardware acceleration**: Assessing performance with cryptographic extensions or FPGA implementations.

- **Protocol integration**: Incorporating lightweight ciphers into IoT protocols such as MQTT, CoAP, or BLE.
- **Side-channel protection**: Investigating resilience to timing, power, and electromagnetic attacks.
- **Automation framework**: Developing reusable embedded benchmarking tools for timing, memory, and throughput profiling.

These directions will deepen the understanding of lightweight cryptographic design and facilitate secure IoT-system development.

Bibliography

[1] Simon Singh, *The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography*. Anchor Books, 2000.

[2] Al-Kindi, "Risāla fī Istikhrāj al-Mu'ammā" (Manuscript on Deciphering Cryptographic Messages), circa 850 CE. In: M. Mrayati, Y. Alam, *Series of Arabic Cryptography Manuscripts*, vol. 1, 1987.

[3] Andrew Hodges, *Alan Turing: The Enigma*. Princeton University Press, 2014.

[4] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, 2001. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf

[5] W. Diffie and M. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976. https://ee.stanford.edu/~hellman/publications/24.pdf

[6] R. Rivest, A. Shamir, and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978. https://people.csail.mit.edu/rivest/Rsapaper.pdf

[7] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin, and C. Vikkelsoe, "PRESENT: An Ultra-Lightweight Block Cipher," in *Proceedings of CHES 2007*, LNCS 4727, pp. 450–466, Springer, 2007. https://link.springer.com/chapter/10.1007/978-3-540-74735-2_31

[8] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, "The SIMON and SPECK Lightweight Block Ciphers," in *Proceedings of the 52nd ACM/EDAC/IEEE Design Automation Conference (DAC)*, pp. 1–6, 2015. https://eprint.iacr.org/2013/404.pdf

[9] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "ASCON v1.2: Lightweight Authenticated Encryption and Hashing," *Journal of Cryptology*, vol. 34, no. 3, pp. 1–42, 2021. https://ascon.iaik.tugraz.at/

[10] National Institute of Standards and Technology, "Lightweight Cryptography: ASCON," *FIPS 203 (Draft)*, 2023. https://csrc.nist.gov/pubs/fips/203/ipd

[11] K. A. McKay and L. Bassham, "Report on Lightweight Cryptography," NISTIR 8114, National Institute of Standards and Technology, 2017. https://nvlpubs.nist.gov/nistpubs/ir/2017/NIST.IR.8114.pdf

[12] J. Guo, T. Peyrin, and A. Poschmann, "The PHOTON Family of Lightweight Hash Functions," in *Proceedings of CRYPTO 2011*, LNCS 6841, pp. 222–239, Springer, 2011. https://link.springer.com/chapter/10.1007/978-3-642-22792-9_13

[13] A. Bogdanov, M. Knežević, G. Leander, D. Toz, K. Varıcı, and I. Verbauwhede, "SPONGENT: A Lightweight Hash Function," in *Proceedings of CHES 2011*, LNCS 6917, pp. 312–325, Springer, 2011. https://link.springer.com/chapter/10.1007/978-3-642-23951-9_21

[14] M. Hell, T. Johansson, and W. Meier, "Grain: A Stream Cipher for Constrained Environments," *International Journal of Wireless and Mobile Computing*, vol. 2, no. 1, pp. 86–93, 2007. https://dl.acm.org/doi/10.1504/IJWMC.2007.013798

[15] C. De Cannière and B. Preneel, "Trivium," in *New Stream Cipher Designs*, LNCS 4986, pp. 244–266, Springer, 2008. https://link.springer.com/chapter/10.1007/978-3-540-68351-3_18

[16] S. Al-Sarawi, M. Anbar, K. Alieyan, and M. Alzubaidi, "Internet of Things (IoT) Communication Protocols: Review," in *2017 8th International Conference on Information Technology (ICIT)*, 2017. https://ieeexplore.ieee.org/document/8079928

[17] Raspberry Pi Foundation, "Raspberry Pi Pico Datasheet," 2021. https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf

[18] Rust Embedded Working Group, "The Embedded Rust Book," 2022. https://docs.rust-embedded.org/book/